國立臺北科技大學
National Taipei University of Technology

資料結構

Data Structure

Lab 07

姓名：廖彥瑄

學號：10810030

Lab07-Ex1.
Convert Infix Expression to Postfix Using a Stack Implemented with Linked List
Code

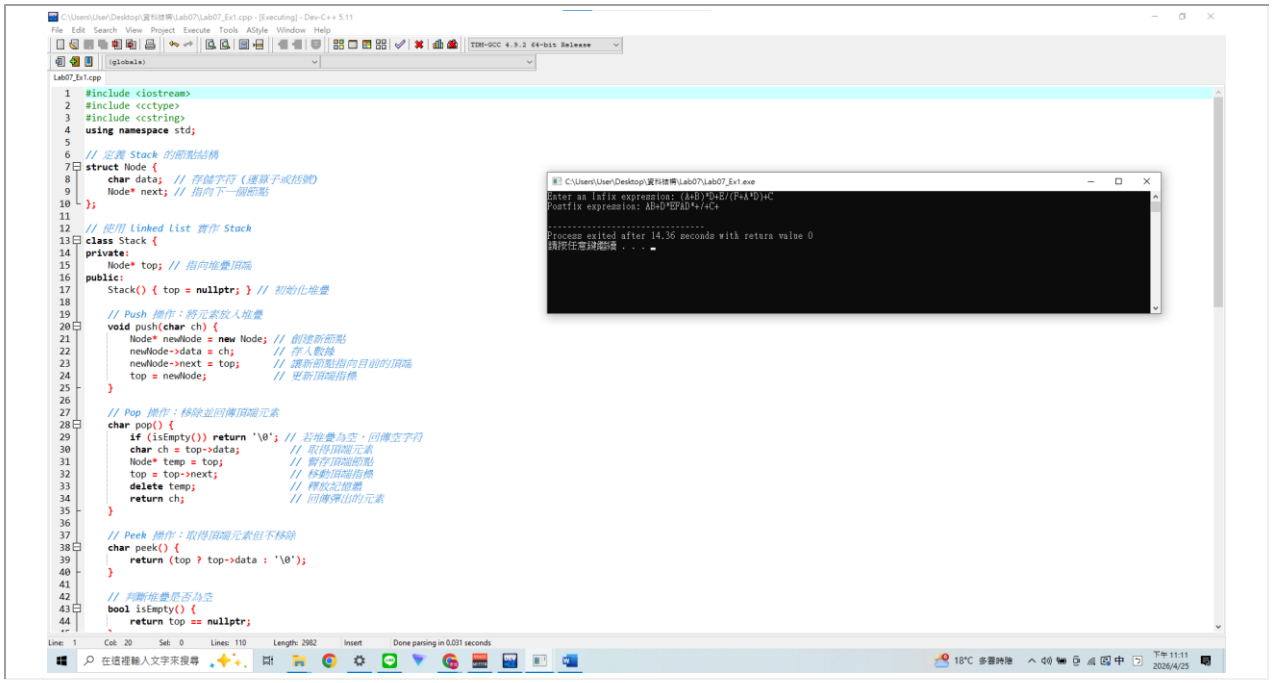
```cpp
#include <iostream>
#include <cctype>
#include <cstring>
using namespace std;

// 定義 Stack 的節點結構
struct Node {
    char data;  // 存儲字符 (運算子或括號)
    Node* next; // 指向下一個節點
};

// 使用 linked list 實作 Stack
class Stack {
private:
    Node* top; // 指向堆疊頂端
public:
    Stack() { top = nullptr; } // 初始化堆疊

    // Push 操作：將元素放入堆疊
    void push(char ch) {
        Node* newNode = new Node; // 創建新節點
        newNode->data = ch;       // 存入數據
        newNode->next = top;      // 讓新節點指向目前的頂端
        top = newNode;            // 更新頂端指標
    }

    // Pop 操作：移除並回傳頂端元素
    char pop() {
        if (isEmpty()) return '\0'; // 若堆疊為空，回傳空字符
        char ch = top->data;         // 取得頂端元素
        Node* temp = top;            // 暫存頂端節點
        top = top->next;             // 移動頂端指標
        delete temp;                 // 釋放記憶體
```

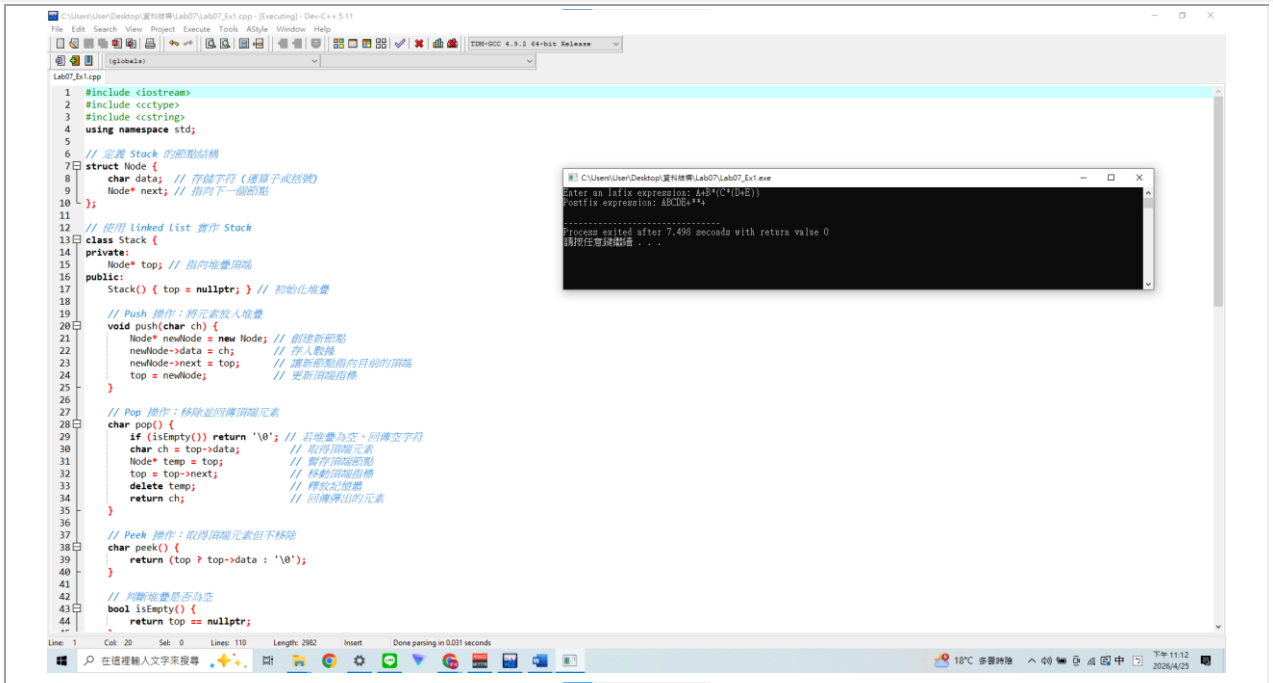
```
        return ch;                  // 回傳彈出的元素
    }

    // Peek 操作：取得頂端元素但不移除
    char peek() {
        return (top ? top->data : '\0');
    }

    // 判斷堆疊是否為空
    bool isEmpty() {
        return top == nullptr;
    }
};

// 判斷運算子(加減乘除) 的優先順序
int precedence(char op) {
        if (op == '+' || op == '-') {
        return 1;//給+和-優先順序1
    }
    if (op == '*' || op == '/') {
        return 2;//給*和/優先順序2
    }
    else {
        return 0;//其他的給優先順序0
    }
}

// 將中序表達式 (infix) 轉換為後序表達式 (postfix)
void InfixToPostfix(const char* infix, char* postfix) {
    Stack stack;
    int j = 0;//用來紀錄放到輸出的哪個位置
    for (int i = 0; infix[i] != '\0'; i++) {
        char ch = infix[i];//放到ch

        if (isalnum(ch)) {
            postfix[j] = ch;//字母和數字直接輸出
            j++;
        }
```

```
        else if (ch == '(') {
            stack.push(ch);//(放進 stack
        }
        else if (ch == ')') {
            while (!stack.isEmpty() && stack.peek() != '(') {
                postfix[j] = stack.pop();//把直到左括號或直到 stack 變空的運算符號都
pop
                j++;
            }

            stack.pop();//(拿掉
        }
        else {
            while (!stack.isEmpty() && stack.peek() != '(' &&
precedence(stack.peek()) >= precedence(ch)) {
                postfix[j] = stack.pop();//當優先順序高的運算符號遇到優先順序低的時候
高的要被輸出
                j++;
            }

            stack.push(ch);//低的放進 stack
        }
    }

    while (!stack.isEmpty()) {
        postfix[j] = stack.pop();//把剩下的都輸出
        j++;
    }

    postfix[j] = '\0';
}

int main() {
    char infix[100], postfix[100];
    cout << "Enter an Infix expression: ";
    cin >> infix; // 輸入中序表達式

    InfixToPostfix(infix, postfix); // 轉換為後序表達式
```

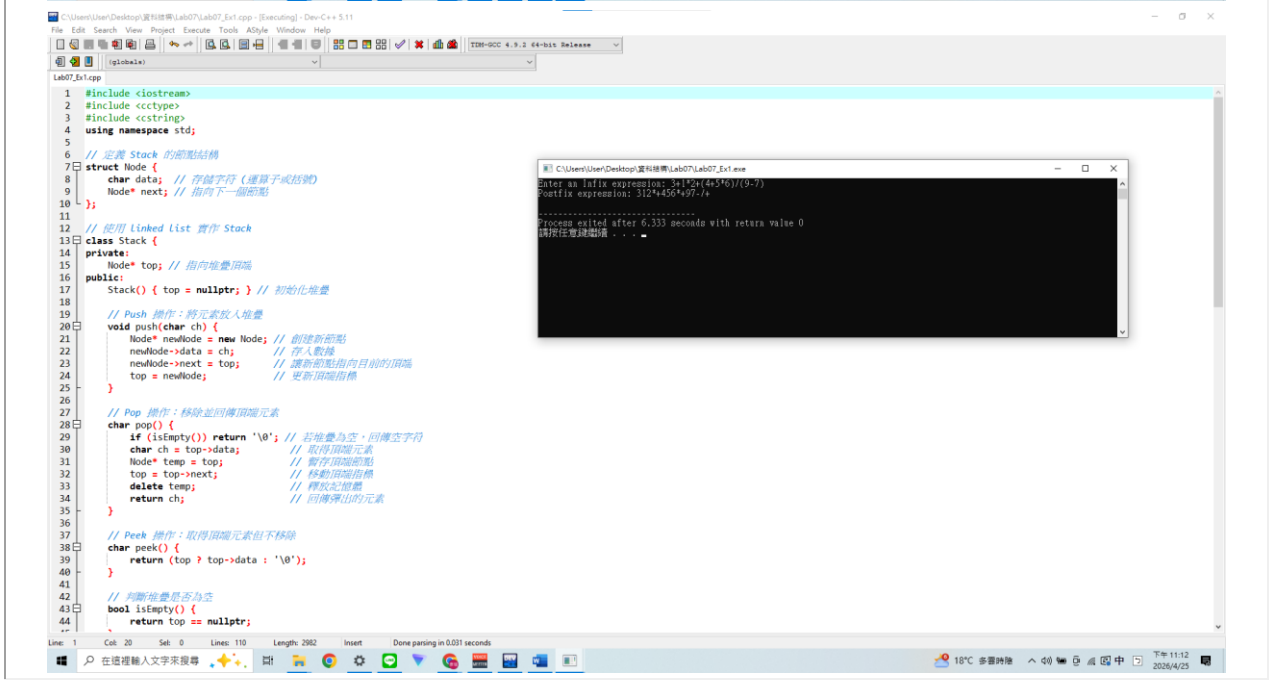
```
    cout << "Postfix expression: " << postfix << endl; // 輸出後序表達式


    return 0;
}
```

Result




C:\Users\User\Desktop\資料結構\Lab07\Lab07_Ex1.cpp - [Executing] - Dev-C++ 5.11
File Edit Search View Project Execute Tools AStyle Window Help
TDM-GCC 4.9.2 64-bit Release
(globals)
Lab07_Ex1.cpp
#include <iostream>
#include <cctype>
#include <cstring>
using namespace std;
// 定義 Stack 的節點結構
struct Node {
char data; // 存儲字符（運算子或括號）
Node* next; // 指向下一個節點
};
// 使用 linked list 實作 Stack
class Stack {
private:
Node* top; // 指向堆疊頂端
public:
Stack() { top = nullptr; } // 初始化堆疊
// Push 操作：將元素放入堆疊
void push(char ch) {
Node* newNode = new Node; // 創建新節點
newNode->data = ch; // 存入數據
newNode->next = top; // 讓新節點指向目前的頂端
top = newNode; // 更新頂端指標
}
// Pop 操作：移除並回傳頂端元素
char pop() {
if (isEmpty()) return '\0'; // 若堆疊為空，回傳空字符
char ch = top->data; // 取得頂端元素
Node* temp = top; // 暫存頂端節點
top = top->next; // 移動頂端指標
delete temp; // 釋放記憶體
return ch; // 回傳彈出的元素
}
// Peek 操作：取得頂端元素但不移除
char peek() {
return (top ? top->data : '\0');
}
// 判斷堆疊是否為空
bool isEmpty() {
return top == nullptr;
C:\Users\User\Desktop\資料結構\Lab07\Lab07_Ex1.exe
Enter an infix expression: A+B*(C*(D+E))
Postfix expression: ABCDE+**+

Process exited after 7.498 seconds with return value 0
請按任意鍵繼續 . . .
Line: 1 Col: 20 Sel: 0 Lines: 110 Length: 2982 Insert Done parsing in 0.031 seconds


C:\Users\User\Desktop\資料結構\Lab07\Lab07_Ex1.cpp - [Executing] - Dev-C++ 5.11
File Edit Search View Project Execute Tools AStyle Window Help
TDM-GCC 4.9.2 64-bit Release
(globals)
Lab07_Ex1.cpp
#include <iostream>
#include <cctype>
#include <cstring>
using namespace std;
// 定義 Stack 的節點結構
struct Node {
char data; // 存儲字符（運算子或括號）
Node* next; // 指向下一個節點
};
// 使用 linked list 實作 Stack
class Stack {
private:
Node* top; // 指向堆疊頂端
public:
Stack() { top = nullptr; } // 初始化堆疊
// Push 操作：將元素放入堆疊
void push(char ch) {
Node* newNode = new Node; // 創建新節點
newNode->data = ch; // 存入數據
newNode->next = top; // 讓新節點指向目前的頂端
top = newNode; // 更新頂端指標
}
// Pop 操作：移除並回傳頂端元素
char pop() {
if (isEmpty()) return '\0'; // 若堆疊為空，回傳空字符
char ch = top->data; // 取得頂端元素
Node* temp = top; // 暫存頂端節點
top = top->next; // 移動頂端指標
delete temp; // 釋放記憶體
return ch; // 回傳彈出的元素
}
// Peek 操作：取得頂端元素但不移除
char peek() {
return (top ? top->data : '\0');
}
// 判斷堆疊是否為空
bool isEmpty() {
return top == nullptr;
C:\Users\User\Desktop\資料結構\Lab07\Lab07_Ex1.exe
Enter an infix expression: 3+1*2+(4+5*6)/(9-7)
Postfix expression: 312*456*+97-/+

Process exited after 6.333 seconds with return value 0
請按任意鍵繼續 . . .
Line: 1 Col: 20 Sel: 0 Lines: 110 Length: 2982 Insert Done parsing in 0.031 seconds

Discussion

Token	stack [0] [1] [2] [3] [4]	-1 Top	Output
(	(	0	
A	(	0	A
+	(+	1	A
B	(+	1	AB
)		-1	AB+
*	*	0	AB+
D	*	0	AB+D
+	+	0	AB+D*
E	+	0	AB+D*E
/	+ /	1	AB+D*E
(	+ / (	2	AB+D*E
F	+ / (	2	AB+D*EF
+	+ / (+	3	AB+D*EF
A	+ / (+	3	AB+D*EFA
*	+ / (+ *	4	AB+D*EFA
D	+ / (+ *	4	AB+D*EFAD
)	+ /	1	AB+D*EFAD*+
+	+	0	AB+D*EFAD*+/+
C	+	0	AB+D*EFAD*+/+C
<eos>		-1	AB+D*EFAD*+/+C+

Token	stack [0] [1] [2] [3] [4] [5]	-1 Top	Output
A		-1	A
+	+	0	A
B	+	0	A B
*	+ *	1	A B
(	+ * (	2	A B
C	+ * (	2	A B C
*	+ * (*	3	A B C
(	+ * (* (	4	A B C
D	+ * (* (	4	A B C D
+	+ * (* (+	5	A B C D
E	+ * (* (+	5	A B C D E
)	+ * (*	3	A B C D E +
)	+ *	1	A B C D E + *
<eos>		-1	A B C D E + * * +

Token	stack [0] [1] [2] [3] [4] [5]	Top (-1)	Output
A		-1	A
+	+	0	A
B	+	0	A B
*	+ *	1	A B
(	+ * (	2	A B
C	+ * (	2	A B C
*	+ * (*	3	A B C
(	+ * (* (	4	A B C
D	+ * (* (	4	A B C D
+	+ * (* (+	5	A B C D
E	+ * (* (+	5	A B C D E
)	+ * (*	3	A B C D E +
)	+ *	1	A B C D E + *
<eos>		-1	A B C D E + * * +